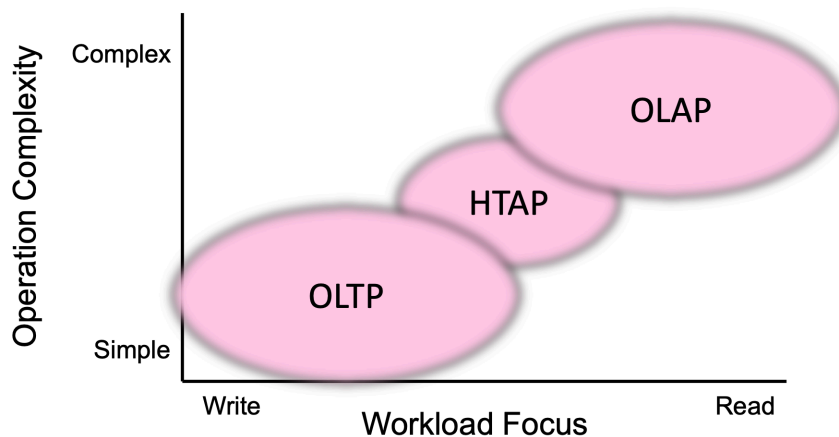


11 OLTP vs. OLAP, Storage Models

▼ OLTP vs OLAP

General:



OLTP:

Online Transaction Processing

- Better for **writing queries and updating the database**
- Simpler databases
- Fast operations that only read/update small amount of data each time
- Interact with specific rows quickly.
- Focuses on simple, fast read/write operations on

OLAP:

Online Analytical Processing

- Better for **reading / analyzing** the database
- More complex databases
- Complex queries that read a lot of data
- Read and analyze lots of rows at once.
- Involves aggregations, summaries, trends, and large-scale analytical operations.

individual or small groups of records

HTAP:

Hybrid Transaction and Analytical Processing

- OLTP and OLAP together in the same database

▼ OLTP Example

```
-- price of a tea
select *
from product
where productname = 'Tea';

-- price of 'fancy' product
-- is unknown
insert into product
values('Fancy', null);

-- update user Donna
update user
set email = 'donna@itu.dk',
password = '***'
where username = 'Donna';
```

▼ OLAP Example

```
-- how many products did
-- each customer buy
select username,
count(*) as count
from purchase
group by username
order by count desc;
```

```
-- cumulative sum of sales
-- per user over time
select *,
sum(price) over
(partition by username
order by purchasetime)
as cum_sales
from sales
order by purchasetime;
```

▼ Storage Models

▼ N-ary Storage Models (NSM)



Store all attributes of tuple in **single file** (tuple by tuple)

- Known as **“row store”**
- Perfect for OLTP, more likely to **access individual entities**

Advantages

- Fast inserts, updates and deletes

Disadvantages

- Bad for scanning large portions of data / subset

- Good memory locality for queries
- OLTP
- Terrible for OLAP access patterns

▼ Decomposition Storage Model (DSM)



Attributes are **stored column by column**

- "Column store"
- Ideal for **OLAP** workloads
- DBMS is responsible for combining/splitting a tuple's attributes when reading/writing

Purchase.dsm

Header				
1	2	3	4	5
Feb 11, 9:55			Feb 12, 10:03	
Feb 12, 10:05			Feb 12, 10:06	
Feb 19, 9:00				
Tea	Small	Small	Large	
Small				
Martin	Martin		Omar	
Omar	Martin			

Advantages

- Good for scanning large amounts of data
- Great for **OLAP** workloads
- Good for example when only looking at the "dates" or "sizes"

Disadvantages

- Slow for point queries, insert, updates, deletes, because it has to read in multiple locations

▼ Efficient Query Execution



Efficient query execution is crucial in cloud-native systems where **compute and storage are separated**.

▼ Pruning Large Sets of Files

Concept:

Data resides in cloud storage, so compute nodes must retrieve files to process queries. Pruning aims to **reduce the number of accessed files**—a key performance booster.



Roughly **50% of execution time** in cloud systems is spent on scanning and filtering.

Technique: Min-Max Indexes (Zone Maps)

- Stored per file, per column.
- Help rule out files during compile time if a file's value range doesn't match the query predicate.

Figure:

Two data layouts:

a			b		
ID	Name		ID	Name	
1	Pinar	File 1	3	Dovile	File 4
2	Veronika		5	Eleni	
3	Dovile		6	Kasper	File 5
4	Martin	File 2	4	Martin	
5	Eleni		1	Pinar	File 6
6	Kasper	File 3	2	Veronika	

- **Fig. 2a:** Data sorted by **ID**, stored in File 1 to 3.
- **Fig. 2b:** Data sorted by **Name**, stored in File 4 to 6.

Example Query:

```
SELECT * FROM users WHERE ID = 5;
```

➡ Only File 2 contains **ID = 5**; others are pruned.

Compile-Time vs. Runtime Pruning

Compile-Time Pruning:

Uses global min-max stats from metadata store.

Runtime Pruning Techniques:

1. **Filter Pushdown:** Use row group/page stats (e.g., in Parquet) to prune sub-file units.
2. **Top-K Pruning:** A **TopK** operator dynamically lowers the boundary, skipping unnecessary rows/files.
3. **Join Pruning:** Send summaries (e.g., Bloom filters or min-max stats) across joins to prune probe side files.



Top-K pruning has shown improvements of up to **100x in query performance**.

▼ Optimizing Data Layouts

Motivation:

Pruning only works if data layout aligns with the access pattern.



If your workload often filters by Name, layout should cluster/sort by Name.

Technique: Clustering

- **Sort data files** to optimize pruning along specific columns ("clustering key").
- Snowflake **allows users to define** this explicitly.

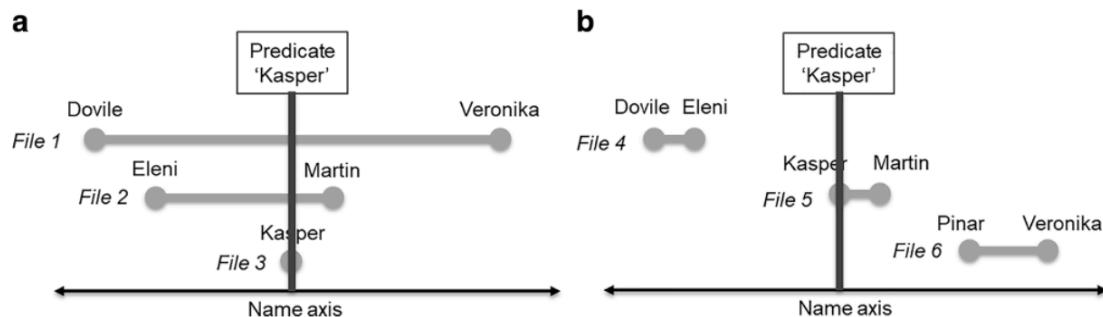
Metric: Clustering Depth

- Measures **how many files overlap** for a given predicate value.



- Lower depth = better pruning = faster queries.

Figure 4:

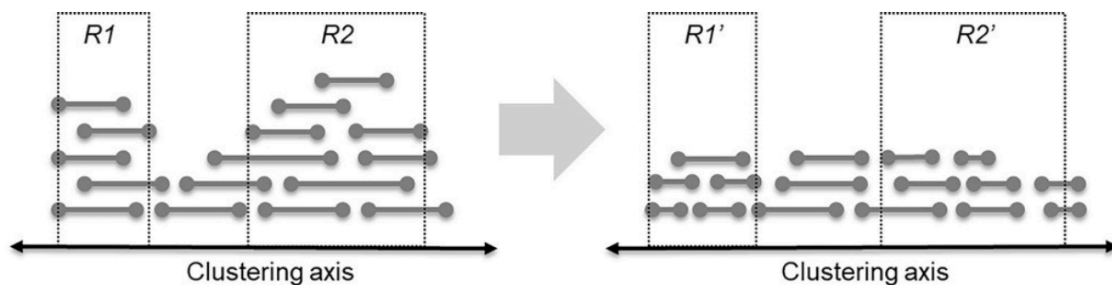


- **4a:** Files 1–3, sorted by `ID` → high clustering depth for `Name`.
- **4b:** Files 4–6, sorted by `Name` → low clustering depth for `Name = 'Kasper'`.

Background Re-Clustering

- A background service sorts sections of tables exceeding a depth threshold.
- Files are replaced with newly clustered ones **without impacting customer workloads** (optimistic commit).

Figure 5:



- Shows how regions R1 and R2 are re-clustered to reduce depth.

▼ Materialization and Caching

Goal:

Reuse previous work to avoid redundant computations.

Reuse Techniques:

1. Result Reuse:

- Return cached results if query and underlying data haven't changed.
- Requires query fingerprinting (hashes) and fast metadata lookup.

2. File Caching:

- Store immutable files locally on SSD.
- Simple and coherent due to immutability.

3. **Min-Max Index Cache:**

- Store frequently accessed stats in compiler memory.

⚡ "Cache warming" can prevent slowdowns during machine restarts.